UNIVERSITY OF OSLO

Assignment– Autumn Semester 2019

Ioanna Maria Lazarou

Data Science for Economists - ECON 4170

Table 1: Shower parameters Table

Parameter	Description
`fLength`	continuous, major axis of ellipse [mm]
`fWidth`	continuous, minor axis of ellipse [mm]
`fSize`	continuous, 10-log of sum of content of all pixels [in number of phot]
`fConc`	continuous, ratio of sum of two highest pixels over fSize [ratio]
`fConc1`	continuous, ratio of highest pixel over fSize [ratio]
`fAsym`	continuous, distance from highest pixel to center, projected onto major axis [mm]
`fM3Long`	continuous, 3rd root of third moment along major axis [mm]
`fM3Trans`	continuous, 3rd root of third moment along minor axis [mm]
`fAlpha`	continuous, angle of major axis with vector to origin [deg]
`fDist`	continuous, distance from origin to center of ellipse [mm]
`class`	g(gamma (signal))and h(hadron (background))

1 Introduction

Data Set: MAGIC Gamma Telescope Data Set

MAGIC (Major Atmospheric Gamma Imaging Cherenkov Telescopes) is a system of two Imaging Atmospheric Cherenkov telescopes situated at the Roque de los Muchachos Observatory on La Palma Canary Islands, at about 2200 m above sea level. MAGIC detects particle showers that enter the atmosphere released by high energy gamma rays, using the Cherenkov radiation. The Cherenkov radiation(of visible to UV wavelengths) gets recorded in the detector, allowing reconstruction of the shower parameters. The goals of the system telescopes are to detect and study primarily photons possibly coming from phenomena such as supernova remnants, accretion of black holes in active galactic nuclei, gamma ray bursts and more.

The data set was generated by a Monte Carlo program to simulate registration of high energy gamma particles in a ground-based atmospheric Cherenkov gamma telescope using the imaging technique [1].

The available information in the data set consists of the shower parameters that can be found in table 1. Depending on the energy of the primary gamma, a total of few hundreds to some 10000 Cherenkov photons get collected, in patterns or as else called the shower image. In that way, it is possible to discriminate statistically those caused by primary gammas(classified as: signal) from the images of hadronic showers initiated by cosmic rays in the upper atmosphere(classified as: background).

A good way to start any data science project is to get a feel for the data. This is just a quick look to see the variable names and expected variable types. Looking at the dimensions of the data is also useful. Data in this project is a data frame of 19020 observations of 11 variables. All variables are numerical, except for one that is categorical. No missing values are recorded in the data set.

2 Background

In this section, we provide background information regarding the major components of our methodology.

2.1 Min-max normalization

Min-max normalization, or feature scaling performs a linear transformation on the original data [2]. It is typically used to bring all values into the range [0,1]. The formula to achieve this is the following:

$$x_{scaled} = \frac{(x - x_{min})}{(x_{max} - x_{min})} \tag{1}$$

It is important that it preserves the relationships among the original data values. There is one disadvantage when we normalize data and that is that standard deviations will then be smaller, fact that can suppress the effect of outliers.

2.2 Training, Validation and Test sets

Data in our model should be split into three different data sets for training and testing purposes. These data sets will consist of a training set, a validation set and a test set [3].

The first set, the training set, is the set of data that will be used to train the model. The model using the data of the training set, will be trained repeatedly and it will continue to learn about the features of this data. After using a training set, we can achieve good performance of our model and have accurate predictions on new unseen data. That means tha the model will be able to make these predictions based on what it learned in its training period.

The validation set is a set of data different from the training set that is used to validate our model during training. This validation process helps give information that it may aid us in adjusting our hyper parameters. So, at the same time, during the training period, the model will be trained on the data in the training set and validating on the data in the validation set. The model will be classifying for all the inputs in the training set. After this classification occurs, the loss will then be calculated and the weights in the model will be adjusted. The classification period can last a lot in terms of time, depending on the size of the data. Also during training, the model will be classifying each input from the validation set as well as it will be doing this classification based only on what it has learned about the data it is being trained on in the training set.

One of the major reasons we need a validation set is to make sure that our model is not overfitting/underfitting to the data in the training set. The idea of overfitting is that our model becomes really great at being able to classify the data in the training set but it is not able to generalize and make accurate classifications based on new data, data that has not trained on. During training if we are validating the model, we can be more confident that our is not overfitting. On the opposite case, if the results on the training data are good but the results on the validation data are lagging behind, then our model is most likely overfitting.

The test set is a distinct one from both the training set and the validation set of data that is used to test the model after the training and validation part. The next step is to use our model for predictions of the output of the data in the test set. The test set should not be labeled as the other two sets. We must not have access to the metrics given during trainings like the loss in the accuracy from each epoch in contrast with the other two sets. The entire goal of having such a model is to be able to classify without knowing what the data is beforehand.

2.3 Feature Selection

In order to achieve better accuracy for our model and great performance with short training times, selecting the right features in the data is very important. The reason for that is that the data features affect greatly on the performance we can achieve when training machine learning models. So, choosing the right variables/features is key when it comes to building predictive models [4]. Irrelevant or partially relevant features can, therefore, negatively impact the performance of the model.

With various feature selection methods, those unneeded, irrelevant and redundant attributes can successfully be removed so that they do not interact negatively with the accuracy of the predictive model. Therefore, by choosing the features that will give as good as possible accuracy while less required data we can create an accurate predictive model. In general, fewer attributes is preferable because first it makes the model less complex, and second, the model is easier to be understood and explained.

2.3.1 Correlation Analysis

One way for choosing the best features for our model is using the correlation matrix. Correlation states how the features are related to each other or to one target variable. It can either be positive or negative. A positive correlation indicates that an increase in one value of feature increases the value of the target variable and a negative correlation indicated that an increase in one value of feature decreases the value of the target variable.

So, after the calculation of the correlation matrix, this feature selection method finds the attributes that are highly correlated, with a set threshold to 0.75, and removes those highly correlated attributes based on the index.

2.3.2 Recursive Feature Elimination (RFE)

Recursive feature elimination (RFE) is a different feature selection method. What it does is that it fits a model and removes the weakest features until the specified by the user number of features is reached. It recursively eliminates a small number of features in each loop. This feature selection method requires a specified number of features to keep and in order to get the ideal number of features, cross-validation is needed. This method scores different feature subsets and selects among them the best scoring ones.

2.3.3 Importance based on Learning Vector Quantization (LVQ)

The importance of features can be estimated from data by building a model. Some methods like decision trees have a built in mechanism to report on variable importance.

For other algorithms, the importance can be estimated using a ROC curve analysis conducted for each attribute. The algorithm constructs an Learning Vector Quantization (LVQ) model with variable importance to be estimated.

2.3.4 Boruta

Boruta is a feature ranking and selection algorithm based on random forests algorithm [5]. The advantage with Boruta is that it clearly decides if a variable is important or not and helps to select variables that are statistically significant.

2.4 Balancing classes

Imbalanced classes are a very usual problem in machine learning classification and a method for dealing with those is the resampling technique. . The problem is that the analogy between the ratio of the number of observations in each class is not proportionate. After the resampling technique, we have an equal ratio of data points for each class. Most machine learning algorithms work best when the number of samples in each class are about equal because in this way, accuracy is maximized and error is reduced. It is wise to try several approaches and decide accordingly which is best way for the problem. There are two resampling techniques, oversample the minority class or undersample the majority class [6].

2.4.1 Undersampling the majority class

Undersampling is simply removing some observations of the majority class. When data is massive, this method is preferred. A disadvantage is that with undersampling, information that may be valuable is removed. Underfitting and poor generalization to the test set are subsequences in this case.

2.4.2 Oversampling the minority class

Oversampling is adding more copies of the minority class and it is preferred with non too massive data. Oversampling takes place before splitting the data and it results in the same number of observations to be present in both the test and train sets. There is also a disadvantage in this method. The model can memorize specific data points and cause, thus, overfitting and poor generalization to the test data.

2.4.3 Synthetic Minority Over-Sampling Technique (SMOTE)

One more method for handling imbalanced data is to generate synthetic samples. SMOTE or Synthetic Minority Oversampling Technique. SMOTE uses a nearest neighbors algorithm to generate new and synthetic data we can use for training our model [7].

2.5 Machine Learning Classifiers

Finding a good machine learning algorithm for a problem that performs the best can be difficult and it demands tuning of its parameters. There are several machine learning

classifiers that can help in classification problems, in which class an observation belongs to [8].

2.5.1 Random Forests (RF)

Random forest algorithm is a one of the most popular and most powerful supervised Machine Learning algorithm in Machine Learning that can perform both regression and classification tasks and has the power to handle large datasets with higher dimensionality [9]. This algorithm creates a forest with a number of decision trees.

In random forests, we grow multiple trees to classify a new object based on attributes. Each tree gives a classification and we save the tree votes for that class. The forest selects the classification that has the majority of votes among the rest trees in the forest. The more trees there are in the forest the more robust the prediction can be. Just like with random forest classifier, the higher the number of trees in the forest gives the high accuracy results.

Many decision trees can deform the random forest. In our problem, multiple trees are grown in order to classify if a new object is caused by primary gamma rays that are classified as gamma signal(g) or initiated by cosmic rays in the upper atmosphere classified as hadron background(h). Each random forest will predict a different outcome or class for the same test feature. For example, if gamma signal is getting highest votes, then the final random forest returns the gamma signal as its predicted target. This concept of voting is known as majority voting.

Random forests are widely known as ensemble machine learning algorithm. A single classifiers on its own is a weak learner. It can be difficult to classify a new object based only in one single classifier. Maybe the information it contains is not enough for a right prediction. But with multiple classifiers we can achieve a strong learner algorithm and thus have improved performance and better accuracy on predictions.

2.5.2 Support Vector Machines (SVM)

Support Vector Machines (SVM) is a supervised machine learning algorithm [10] used for classification and regression problem. Unlike traditional regression approaches such as linear regression, it uses a technique called the kernel trick to transform the data and then based on these transformations it finds an optimal boundary between the possible outputs. Throughout the years, SVM have become extremely popular in the machine learning community.

3 Performance Evaluation

In this section, we present the results from the classification analysis problem and discuss our findings.

3.1 Classification Error Metrics

First, we provide a brief description of the classification error metrics we use to evaluate the performance of the classification models.

3.1.1 Confusion Matrix

A confusion matrix is a contingency table of correct and incorrect classifications. Correct classifications are on the diagonal of the confusion matrix. The items on the diagonals are also called the true positives and true negatives. The off-diagonals are called the false positives versus the false negatives.

Several measures can be derived from the confusion matrix as follows.

- Accuracy: the proportion of the total number of correct predictions.
- Sensitivity (or Recall): the number of true positives divided by the number of true positives plus false negatives.
- Specificity: the proportion of actual negative cases identified correctly.
- Precision: the number of true positives divided by the number of true positives plus false positives.
- F1 Score: harmonic mean of precision and recall

3.1.2 Receiver Operator Characteristic (ROC) and Area Under Curve (AUC)

A Receiver Operator Characteristic (ROC) curve is a graphical plot used to show the diagnostic ability of binary classifiers. A ROC curve is constructed by plotting the true positive rate (TPR) against the false positive rate (FPR). So, The true positive rate is the fraction of observations that were correctly predicted to be positive out of all positive observations. Similarly, the false positive rate is the fraction of observations that are not predicted correctly to be positive out of all negative observations. In our testing, the true positive rate is the rate in which an observation is correctly identified to test positive for the class gamma signal or hadron background in question.

The ROC curve shows the a balance achieved between between sensitivity and specificity. Classifiers that give curves closer to the top left corner of the graph have generally better performance and the closer the curve comes to the 45-degree diagonal of the ROC space, the less accurate the test is indicated to be. One way of comparing different classifiers is to calculate the area under the ROC curve which performs well as a measure of predictive accuracy. This area is named as AUC.

3.2 Classification Analysis Results

An abstract flowchart of the proposed methodology can be depicted in Figure 2. All blocks have been extensively described in Section 2 along the available methods. To ensure a robust and efficient model, we try different combinations and report a list of the results in Tables 2 (training set) and 3 (testing set). Since the number of available combinations is quite big[1], we select only a subset of them seen in table 1. Due to the fact that the feature

[1]There are two options when it comes to normalize our data (min-max normalization or no normalization), four feature selection approaches (Correlation Analysis, RFE, Importance based on LVQ and Boruta), three balancing techniques (undersampling, oversampling and SMOTE) and two classifiers (Random Forests and Support Vector Machines). Therefore the total number of available combinations is 48.

Table 2: Accuracy and Kappa on the training set.

	1	**2**	**3**	**4**	**5**	**6**	**7**	**8**
Accuracy %	0.818	0.799	0.849	0.926	0.887	0.781	0.776	0.786
Kappa	0.637	0.599	0.698	0.852	0.774	0.562	0.553	0.572

Table 3: Classification Metrics on the testing set.

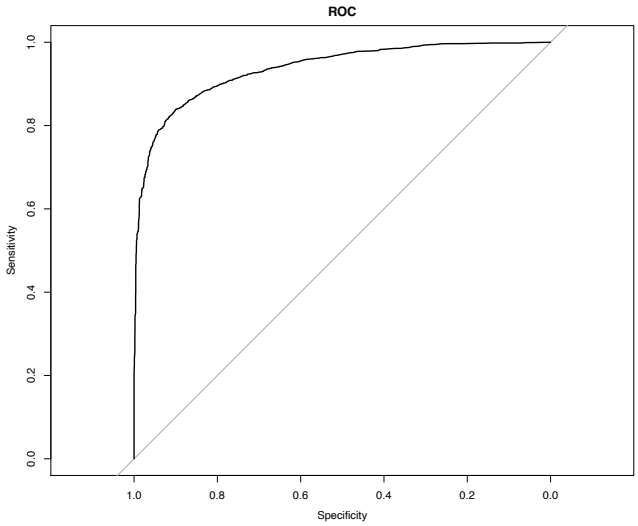
	1	**2**	**3**	**4**	**5**	**6**	**7**	**8**
Accuracy %	0.831	0.795	0.844	0.866	0.782	0.776	0.786	0.791
Sensitivity %	0.853	0.808	0.883	0.893	0.858	0.813	0.823	0.826
Specificity %	0.808	0.782	0.804	0.840	0.705	0.739	0.750	0.756
Precision %	0.816	0.788	0.818	0.848	0.744	0.757	0.767	0.772
Recall %	0.853	0.808	0.883	0.893	0.858	0.813	0.823	0.826
F1 Score	0.834	0.798	0.849	0.870	0.797	0.784	0.794	0.798
AUC	0.914	0.881	0.913	0.936	0.860	0.845	0.857	0.861

space consists of attributes with different scales, we always utilize min-max normalization to transform the values into the [0,1] range.

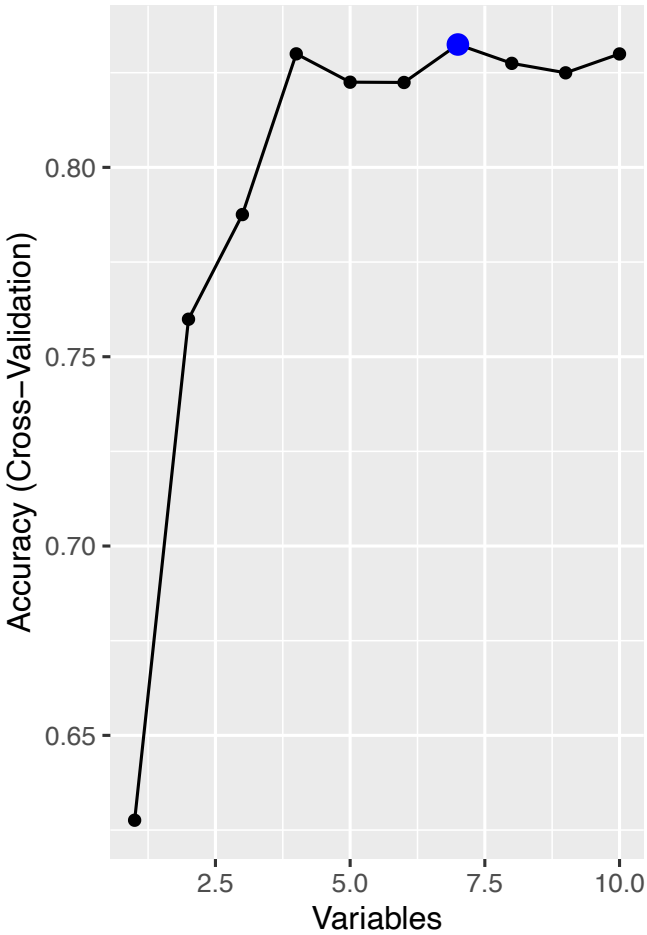
We observe that configuration 4 (min-max normalization, RFE, SMOTE and Random Forests) outputs the best accuracy % (0.926) and kappa (0.852) on the training set. We further see that the nominal model performs well on the testing set with an accuracy of 0.866 showing good generalization capabilities. Figure 1a shows the ROC curve of the model while figure 1b depicts the accuracy results of the different feature subset sizes as reported by the RFE approach. We observe that we get the highest accuracy if we select 8 out of 10 features to include in the model. Last, figure 5 shows the correctly classified number of samples for each of the two classes (`g` and `h` respectively). The framework also provides for automatic tuning of the classifier hyperparameters using a random search scheme (cross validation and multiple repeats) that could lead to a significant rise in the performance.

Table 4: Tested configurations of the classification methodology.

	Normalization	**Feature Selection**	**Balancing**	**Classifier**
1	`min-max`	`Correlation`	`Undersampling`	`Random Forests`
2	`min-max`	`Correlation`	`Oversampling`	`Support Vector Machines`
3	`min-max`	`RFE`	`Undersampling`	`Support Vector Machines`
4	`min-max`	`RFE`	`SMOTE`	`Random Forests`
5	`min-max`	`LVQ`	`Oversampling`	`Random Forests`
6	`min-max`	`LVQ`	`SMOTE`	`Support Vector Machines`
7	`min-max`	`Boruta`	`Undersampling`	`Random Forests`
8	`min-max`	`Boruta`	`SMOTE`	`Support Vector Machines`



(a)



(b)

Figure 1: (a) ROC curve (b) Feature Selection with RFE

Table 5: Confusion Matrix.

P/R	g	h
g	2422	432
h	290	2280

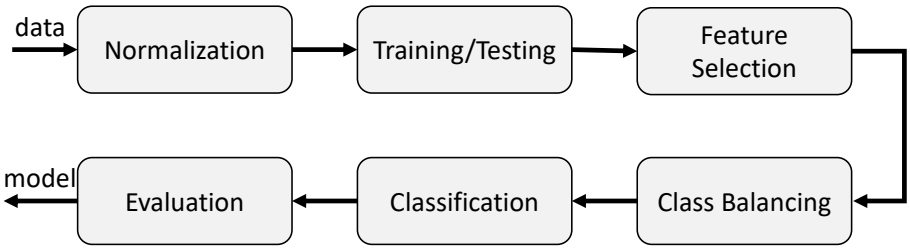


Figure 2: An abstract flowchart of the methodology we follow throughout this project.

4 Conclusions

In this project, we present a classification framework to discriminate statistically those caused by signals of primary gammas from the background, images of hadronic showers initiated by cosmic rays in the upper atmosphere. The proposed methodology includes preprocessing (normalization/handling inbalanced classes/feature selection) methods that prepare the data for classification using machine learning. Results reveal that we can classify the two groups with an accuracy of 0.866 on the testing set. The selected model shows good generalization capabilities as reported by a list of classification error metrics, such as Precision, Recall, F1 Score, etc., and it could be used for classification with new measurement points.

5 Appendix

We attach the R code below used for the classification analysis problem. Dataset can be found here: `https://archive.ics.uci.edu/ml/datasets/MAGIC+Gamma+Telescope` .

```
# load the required R libraries (install the respective packages if
# needed)

library(mlbench)
library(caret)
library(tidyverse)
library(dplyr)
library(Boruta)

library(Metrics)
library(pROC)

source('classification_magic.R')
```

```r
# Read data (comma separated)
data = read.table('magic04.data',sep = ',')

# Rename columns
colnames(data) = c('fLength','fWidth','fSize','fConc','fConc1','fAsym',
                       'fM3Long','fM3Trans','fAlpha','fDist','class')

# Check for missing (NA) values
sapply(data, function(x) sum(is.na(x)))
# No missing values for any of the features involved
# We assume that all reported values for each of the features are
# within the valid range

# For testing purposes (Pick 500 observation in a random fashion)
data = data[sample(nrow(data), 500), ]

classification_magic(data, fs = 2, balance = 3, classifier = 1)

classification_magic <- function(data, normalization = 1,
        split = 0.80, fs = 1, balance = 1, classifier = 1, tune = 0) {

        # Min-max normalization
        if (normalization == 1) {
                data_normalized <- as.data.frame(lapply
                        (data[,c(1:(dim(data)[2]-1))], normalized))
                data_normalized = cbind(data_normalized,
                        data[,dim(data)[2]])
                colnames(data_normalized)[dim(data)[2]] = 'class'
                data = data_normalized
        }

        # Set the split threshold
        sep <- floor(split * nrow(data))

        # Perform data sampling to improve generalization
        train_ind <- sample(seq_len(nrow(data)), size = sep)

        train <- data[train_ind, ]
        test <- data[-train_ind, ]
        # dim(train)
        # dim(test)

        # Feature Selection
        if (fs == 1) { # Correlation
                # Calculate correlation matrix
                # (except the target/categorical value)
                correlationMatrix <- cor(train[,1:(dim(train)[2]-1)])
                # Summarize the correlation matrix
```

```
        print(correlationMatrix)
        # Find attributes that are highly corrected
        # - set threshold to 0.75 - modify on demand
        highlyCorrelated <- findCorrelation(correlationMatrix,
         cutoff=0.75)
        # Remove highly correlated attributes based on the index
        train = train[,-highlyCorrelated]
        test = test[,-highlyCorrelated]
} else if (fs == 2) { # Recursive Feature Elimination (RFE)
        # Define the control using a random forest selection
        # function
        control <- rfeControl(functions=rfFuncs, method="cv",
                number=3)
        # Run the RFE algorithm
        results <- rfe(train[,1:(dim(train)[2]-1)],
                train[,dim(train)[2]],
                sizes=c(1:(dim(train)[2]-1)), rfeControl=control)
        # Summarize the results
        print(results)
        # List the chosen features
        predictors(results)
        # plot the results
        ggplot(results, type=c("g", "o")) + ggsave('rfe.pdf')

        train_t = train %>% select(predictors(results))
        test_t = test %>% select(predictors(results))
        train = cbind(train_t,train[,dim(train)[2]])
        colnames(train)[dim(train)[2]] = 'class'
        test = cbind(test_t,test[,dim(test)[2]])
        colnames(test)[dim(test)[2]] = 'class'
} else if (rf == 3) { # Importance based on Learning Vector
# Quantization (LVQ)
        # Prepare training scheme
        control <- trainControl(method="repeatedcv", number=3,
         repeats=3)
        # train the model
        model <- train(class~., data=train, method="lvq",
         preProcess="scale", trControl=control)
        # estimate variable importance
        importance <- varImp(model, scale=FALSE)
        # summarize importance
        print(importance)
        # plot importance
        png('importance.png')
        plot(importance)
        dev.off()

        features = order(importance[[1]][1]$g,decreasing = TRUE)
```

```
        train_t = train[,features[1:3]]
        test_t = test[,features[1:3]]
        train = cbind(train_t,train[,dim(train)[2]])
        colnames(train)[dim(train)[2]] = 'class'
        test = cbind(test_t,test[,dim(test)[2]])
        colnames(test)[dim(test)[2]] = 'class'
} else { # Boruta
        boruta_output <- Boruta(class ~ ., data=train, doTrace=0)
        names(boruta_output)

        # Get significant variables including tentatives
        boruta_signif <- getSelectedAttributes(boruta_output,
         withTentative = TRUE)
        print(boruta_signif)

        # Do a tentative rough fix
        roughFixMod <- TentativeRoughFix(boruta_output)
        boruta_signif <- getSelectedAttributes(roughFixMod)
        print(boruta_signif)

        # Variable Importance Scores
        imps <- attStats(roughFixMod)
        imps2 = imps[imps$decision != 'Rejected', c('meanImp',
         'decision')]
        head(imps2[order(-imps2$meanImp), ])  # descending sort

        # Plot variable importance
        png('boruta.png')
        plot(boruta_output, cex.axis=.7, las=2, xlab="",
                main="Variable␣Importance")
        dev.off()

        train_t = train[,rownames
        (head(imps2[order(-imps2$meanImp), ]))]
        test_t = test[,rownames
        (head(imps2[order(-imps2$meanImp), ]))]
        train = cbind(train_t,train[,dim(train)[2]])
        colnames(train)[dim(train)[2]] = 'class'
        test = cbind(test_t,test[,dim(test)[2]])
        colnames(test)[dim(test)[2]] = 'class'
}

# Balancing classes
if (balance == 1) { # undersampling the majority class
        class1 = train[train$class=='g',]
        class2 = train[train$class=='h',]

        class1_under = class1[1:dim(class2)[1],]
        train_balanced = rbind(class1_under,class2)
```

```
        class1 = test[test$class=='g',]
        class2 = test[test$class=='h',]

        class1_under = class1[1:dim(class2)[1],]
        test_balanced = rbind(class1_under,class2)
} else if (balance == 2) { # oversampling the minority class
 by replicas
        class1 = train[train$class=='g',]
        class2 = train[train$class=='h',]

        # Find how many samples I need to match the majority class
        samples_to_generate = dim(class1)[1] - dim(class2)[1]
        # Select samples_to_generate random samples
        # from the minority class
        random_samples = sample_n
        (class2,size = samples_to_generate)
        class2_over = rbind(class2,random_samples)

        train_balanced = rbind(class1,class2_over)

        class1 = test[test$class=='g',]
        class2 = test[test$class=='h',]

        # Find how many samples I need to match the majority class
        samples_to_generate = dim(class1)[1] - dim(class2)[1]
        # Select samples_to_generate random samples from the
        # minority class
        random_samples = sample_n(class2,size =
                samples_to_generate)
        class2_over = rbind(class2,random_samples)

        test_balanced = rbind(class1,class2_over)
} else { # SMOTE
        train_balanced <- SMOTE(class ~ ., train, perc.over = 100)
        test_balanced <- SMOTE(class ~ ., test, perc.over = 100)
}
train = train_balanced
test = test_balanced

# Machine Learning Classifiers

# Define control (tuning available)
if (tune == 0) {
        control <- trainControl(method="repeatedcv",
                number=5, repeats=3, classProbs = TRUE)
} else {
        control <- trainControl(method="repeatedcv",
                number=5, repeats=3, classProbs = TRUE,
```

```
                search = 'random')
}

if (classifier == 1) { # Random Forests
        metric <- "Accuracy"
        mtry <- sqrt(ncol(train))
        tunegrid <- expand.grid(.mtry=mtry)
        if (tune == 0) {
                model <- train(class~., data=train, method="rf",
                 metric=metric,
                 tuneGrid=tunegrid, trControl=control)
        } else {
                model <- train(class~., data=train, method="rf",
                 metric=metric, trControl=control, tuneLength=5)
        }
} else { # Support Vector Machines
        model <- train(class ~., data = train,
                method = "svmRadial", trControl=control,
                preProcess = c("center", "scale"))
}
print(model)

predictions = predict(model, newdata = test)
predictions_p <- predict(model, newdata = test, type = "prob")

cm = confusionMatrix(predictions, test$class)

ROC <- roc(test$class, predictions_p[,2])
ROC_auc <- auc(ROC)

plot(ROC, col = "black", main = "ROC")

# print the performance of each model
message("Accuracy␣model␣%:␣", format(model$results$Accuracy,
        digits = 4))
message("Kappa␣model␣%:␣", format(model$results$Kappa,digits = 4))

message("Accuracy␣%:␣", format(cm$overall[1],digits = 4))
message("Sensitivity␣%:␣", format(cm$byClass[1],digits = 4))
message("Specificity␣%:␣", format(cm$byClass[2],digits = 4))
message("Precision␣%:␣", format(cm$byClass[5],digits = 4))
message("Recall␣%:␣", format(cm$byClass[6],digits = 4))
message("F1␣%:␣", format(cm$byClass[7],digits = 4))
message("Area␣under␣curve:␣", format(ROC_auc,digits = 4))

message('Confusion␣matrix:␣')
print(cm$table)

# heatmap(cm$table, col = rainbow(3),xlab = 'Reference',
```

```
                ylab = 'Predicted')
}

normalized <- function(x) { (x- min(x))/(max(x) - min(x)) }
```

References

[1] D. Dua and C. Graff, "UCI machine learning repository," 2017. [Online]. Available: http://archive.ics.uci.edu/ml

[2] S. Patro and K. K. Sahu, "Normalization: A preprocessing stage," *arXiv preprint arXiv:1503.06462*, 2015.

[3] H. Adeli, "Machine learning-neural networks, genetic algorithms and fuzzy systems," *Kybernetes*, 1999.

[4] P. Harrington, *Machine learning in action*. Manning Publications Co., 2012.

[5] M. B. Kursa, W. R. Rudnicki *et al.*, "Feature selection with the boruta package," *J Stat Softw*, vol. 36, no. 11, pp. 1–13, 2010.

[6] G. E. Batista, R. C. Prati, and M. C. Monard, "A study of the behavior of several methods for balancing machine learning training data," *ACM SIGKDD explorations newsletter*, vol. 6, no. 1, pp. 20–29, 2004.

[7] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "Smote: synthetic minority over-sampling technique," *Journal of artificial intelligence research*, vol. 16, pp. 321–357, 2002.

[8] F. Pereira, T. Mitchell, and M. Botvinick, "Machine learning classifiers and fmri: a tutorial overview," *Neuroimage*, vol. 45, no. 1, pp. S199–S209, 2009.

[9] L. Breiman, "Random forests," *Machine learning*, vol. 45, no. 1, pp. 5–32, 2001.

[10] B. Scholkopf and A. J. Smola, *Learning with kernels: support vector machines, regularization, optimization, and beyond*. MIT press, 2001.